

Notification Oriented Paradigm Applied to Ambient Assisted Living Tool

R. N. Oliveira, V. Roth, A. F. Henzen, J. M. Simão, E. C. G. Wille and P. Nohama

Abstract— The rising in life expectancy is reflected in an increase in the elderly population and numerous actions become necessary to ensure the elderly a proper quality of life and independence in performing daily tasks. One of the applications of the Internet of Things technology (IoT) is the creation of smart environments with interactive devices allowing integration between the physical environment and the individual. A viable application of intelligent environments is the Ambient Assisted Living (AAL), which can enable elderly and people with some kind of limitations be assisted in their daily routine, independently and safely. In this context, in order to provide support technology, this research aims to evaluate the adhesion of Notification Oriented Paradigm (NOP) to IoT. NOP provides means for the development of robust systems, distributed, consistent, and rule-based systems for smart environments. Thus, to verify this adherence, it was created an application where is possible to manage rules and sensors dynamically and in real time, besides to enable notifications between sensors connected in different environments. This application can run on a microcomputer Raspberry Pi that simulates the rules and sensors in a virtual environment, acting as a simulator. The result of the simulation was evaluated by measuring the execution time of a set of rules notified among different environments. The results achieved for execution time, in the range of milliseconds, are sufficient for the demands required by systems in real scenarios, thus demonstrating the effectiveness of the intelligent systems developed by means of NOP.

Keywords— Ambient Assisted Living, Notification Oriented Paradigm, Smart Environments to Elderly.

I. INTRODUÇÃO

OS AVANÇOS na medicina têm aumentado a expectativa de vida da população. De acordo com as Nações Unidas, aproximadamente 20% da população mundial terá 60 anos ou mais em 2050 [1]. Entretanto, o avanço da idade é, geralmente, responsável por limitações motoras e sensoriais (visuais, auditivas ou táteis), além do surgimento de afecções crônicas. Este cenário cria vários desafios para a sociedade, entre os quais, ampliação da assistência médico-hospitalar, elevação subsequente de gastos com saúde e escassez de cuidadores de idosos.

Não obstante, o envelhecimento da população deve ser encarado como uma oportunidade para se viver mais e melhor. Para tanto, algumas ações são necessárias para garantir ao idoso a qualidade de vida adequada e sua independência na execução de atividades cotidianas. Dentre elas, desponta a autonomia do idoso em seu domicílio, proporcionada pela computação senciente. Esta baseia-se no monitoramento de um ambiente por meio de sensores e tomadas de decisões de acordo com as mudanças nesse ambiente e voltadas ao bem-estar do ser humano [2].

Os ambientes inteligentes, no âmbito da computação senciente, permitem a interação inteligente e natural entre indivíduo e o ambiente físico [3]. Uma das suas aplicações é o desenvolvimento de sistemas de assistência à autonomia no domicílio (*Ambient Assisted Living* - AAL). O AAL tem por objetivo principal possibilitar que pessoas com algum tipo de limitação motora e/ou sensorial sejam auxiliadas em sua rotina diária, propiciando um estilo de vida independente e seguro o mais duradouro possível, dentro de seus ambientes pessoais [4].

Os sistemas AAL devem ser entidades dinâmicas que possibilitem adições de novos sensores, interpretação de novos comportamentos e execução de novas regras conforme o perfil de cada pessoa presente em uma determinada situação e contexto, sendo que isto exige complexidade maior na codificação desses sistemas. A utilização do Paradigma Orientado a Notificações – PON (ou, em inglês, *Notification Oriented Paradigm* – NOP), pode minimizar esta complexidade.

O PON pode minimizar tal complexidade tendo em vista que seu foco é a execução do programa por meio de notificações entre as entidades, de forma pontual e seletiva, eliminando a necessidade de execução sequencial, predominante nos atuais paradigmas [5][6][7]. Aliado a isso, o PON proporciona a criação, de maneira simples, por programação orientada a regras, de aplicações distribuídas, consistentes e robustas; características estas que são requisitos na computação senciente, particularmente na AAL [8].

Assim sendo, este artigo tem por objetivo avaliar a aderência do PON aos requisitos dos sistemas AAL a partir da criação de conjuntos de regras a serem executadas tanto em uma simulação de ambiente como em microcomputadores com seus respectivos sensores e atuadores.

II. FUNDAMENTAÇÃO

Nesta seção, apresentam-se os fundamentos dos ambientes inteligentes e sua aplicação na assistência à autonomia no domicílio para idosos, além dos fundamentos do Paradigma Orientado a Notificações (PON).

R. N. Oliveira, UTFPR – Universidade Tecnológica Federal do Paraná, Curitiba-PR, rodrigo@gl2.com.br.

V. Roth – UTFPR, valmirroth@gmail.com.

A. F. Henzen – UTFPR, alexandrehenzen@hotmail.com.

J. M. Simão – UTFPR, jeansimao@utfpr.edu.br.

E.C.G. Wille – UTFPR, ewille@utfpr.edu.br.

P. Nohama – UTFPR, percy.nohama@gmail.com.

Assistência à autonomia no domicílio - Ambient Assisted Living - (AAL)

A funcionalidade de um ambiente inteligente pode ser descrita como um ciclo composto por uma etapa de monitoramento do estado do ambiente, seguido pelo seu processamento para alcançar um objetivo específico ou antecipar respostas a possíveis ações, até a atuação sobre o ambiente para alterar seu estado. O monitoramento ocorre por meio da coleta de dados sensoriais, gerados por dispositivos distribuídos em uma rede de sensores que fornece informações para o sistema tomar decisões de como alterar o ambiente [3].

Várias técnicas de inteligência artificial (IA) podem ser utilizadas em ambientes inteligentes, tais como representação do conhecimento, aprendizado de máquina, inteligência computacional, planejamento, reconhecimento de fala, linguagem natural, visão computacional, robótica e sistemas multiagentes. Essas técnicas de IA colaboram para uma análise mais complexa e uma ação mais precisa no ambiente [2].

Os sistemas AAL, quando desenvolvidos como ambientes inteligentes, podem ser usados na prevenção, tratamento e melhora das condições de saúde dos idosos. Sua aplicação pode ocorrer em atividades de monitoramento das condições de saúde, notificações e controle no uso de medicamentos, detecção de quedas, monitoramento de segurança, auxílio nas tarefas cotidianas, facilidade na comunicação do idoso com familiares e enfermeiros, e até mesmo na geração e acompanhamento de um diário para indivíduos com demência [4].

Um dos pontos fundamentais na concepção de sistemas AAL está relacionado ao *design* da aplicação uma vez que a maioria dos usuários teriam alguma limitação, e.g. são idosos com dificuldades na visão e na coordenação motora. É importante projetar uma interface com botões e letras grandes e com o uso de figuras intuitivas, de forma que seja a mais simples e amigável possível para se operar.

Os sensores vestíveis, que monitoram as condições de saúde do usuário, não devem criar nenhum tipo de desconforto ou limitação nos movimentos. Uma alternativa é a utilização de dispositivos que o idoso já esteja familiarizado; por exemplo, celulares, *tablets* ou relógios e jóias inteligentes. Não obstante, é evidente a resistência da maioria dos idosos em relação à tecnologia, sendo necessária uma preocupação em desenvolver soluções que exerçam seu papel com a menor dependência de ações diretas desses usuários.

Não obstante, o sistema AAL jamais deve substituir completamente a necessidade de cuidado humano; pois, nesse caso, poderia resultar inclusive em isolamento do usuário [4].

Paradigma Orientado a Notificações - PON

Os principais paradigmas atuais podem ser classificados em imperativos e declarativos. O imperativo engloba as abordagens procedimentais e orientadas a objetos. O declarativo, por sua vez, envolve abordagens lógicas e funcionais. Em ambos os casos, há a predominância de inferência (implícita ou explícita) orientada a buscas, que tem por finalidade verificar valores de elementos passivos (por exemplo variáveis), para testar expressões lógicas/causais (*if-*

else, por exemplo), criando, assim, avaliações repetitivas e desnecessárias no código [5][6].

Uma nova técnica de programação denominada Paradigma Orientado a Notificações (PON) foi proposta por Simão [6][7], inicialmente, como uma solução de controle discreto de manufatura com uma nova abordagem no sistema de inferência [9], sendo posteriormente, evoluída como um novo paradigma de programação.

A essência do PON está no seu sistema de inferência composto por entidades desacopladas que colaboram por meio de notificações precisas [6][7][9]. Isto resolve o problema da centralização e redundância causados pela atual abordagem de processamento lógico causal dos paradigmas atuais, problema esse que leva à subutilização da capacidade de processamento e acoplamento de código [6][7].

O PON permite a construção de softwares melhores do ponto de vista de um código otimizado e distribuído [6][7], pois sua estrutura otimiza a utilização dos recursos de processamento melhorando o desempenho da aplicação, com a capacidade nativa da utilização de vários núcleos de processamento (*multi-core*) e aplicações distribuídas de forma geral [10][11].

Isto considerado, a Fig. 1 apresenta um diagrama de classes em UML, com as entidades do PON e seus relacionamentos. A *Condition* (Condição) está vinculada a entidades relaciona à decisão da *Rule* (Regra), enquanto a *Action* (Ação) está relacionada às entidades referentes à execução da *Rule* e dos elementos envolvidos [8].

Os elementos avaliados no PON são representados por uma entidade chamada *Fact Base Element* (FBE – Elemento da Base de Fatos), que é composta por um ou mais *Attributes* (Atributos) que possuem a capacidade de notificar as *Rules* interessadas quando seus valores sofrem uma mudança. Esta notificação ocorre em uma entidade chamada *Premise* (Premissas), ou seja, apenas as *Premises* que desejam saber o atual valor do *Attribute* receberão a notificação da mudança de seu valor.

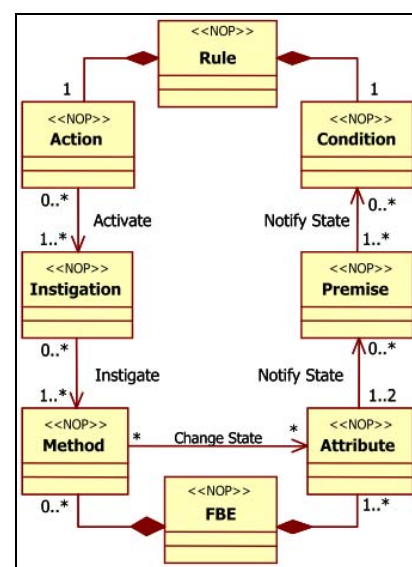


Figura 1. Diagrama de classes do PON [7][8].

O processo de inferência no PON, que ocorre por meio das notificações entre as entidades, é disparado a cada mudança do

valor de um *Attribute* da *FBE*. O *Attribute* apenas notifica o valor alterado a cada *Premise* relacionada a ele. Consequentemente, cada *Premise*, cujo valor lógico altere-se (em decorrência da notificação de *Attribute*), notifica tal modificação de valor lógico apenas sobre as *Conditions* das *Rules* relacionadas e pertinentes são notificadas por esta modificação [8].

Quando as *Premises* que compõem uma *Condition* atendem-na, a *Condition* notifica a respectiva *Rule* para ser aprovada. Cada *Rule* aprovada ativa sua *Action* disparando a execução das *Instigations* (Instigações). Estas, por sua vez, disparam a execução dos *Methods* (Métodos) [8].

Com a colaboração entre as entidades por meio das notificações pontuais e seletivas, o PON evita tanto a redundância temporal (avaliação de expressões lógicas/causais desnecessárias), quanto a redundância estrutural (repetição de expressões lógicas), presentes nas linguagens imperativas.

Além de resolver os problemas de desempenho, o PON é naturalmente aderente ao desenvolvimento de sistemas paralelos e distribuídos, devido ao seu baixo acoplamento de entidades. Em suma, não há diferença se a notificação entre as entidades está ocorrendo no mesmo computador ou em computadores que estão em redes diferentes. Nada impede que a entidade *FBE* esteja sendo executada em um computador e ao ter o valor de um *Attribute* alterado, este notifique a *Premise* que esteja em outro computador. Outra vantagem deste desacoplamento é a capacidade das notificações ocorrerem em paralelo utilizando, assim, a estrutura de multiprocessadores comum nos computadores atuais [8].

As primeiras materializações do PON ocorreram na forma de um *framework*, desenvolvido sobre C++, onde o desenvolvedor só precisa implementar as *FBEs*, com seus *Attributes* e *Methods*, e a criar as regras a partir de uma interface sem se preocupar sobre a criação de ligações para o processo de notificações entre as entidades do PON, que ocorre em segundo plano no âmbito do *framework* [8].

Atualmente, existem *frameworks* do PON em várias linguagens como: C++, Java, C# e VHDL. Além deles, há também, um compilador e uma linguagem própria para o PON, denominada LingPON [12], que permite gerar código para uma das versões do *framework* PON, bem como para código mais de baixo nível.

Para exemplificar uma aplicação em PON, considere-se uma *Rule* do ambiente que liga a iluminação e aciona um aviso sonoro quando há uma pessoa nele, e sendo a distância dessa pessoa em relação à porta menor ou igual a 50 centímetros. Nesse caso, haverá uma *FBE* para o ambiente com *Attributes* para controle das pessoas presentes, da distância até a porta, da iluminação, e do acionamento da sirene (aviso sonoro).

Tal *Rule* é composta por uma *Condition* vinculada a duas *Premises*: a primeira para verificar se existe pessoa no ambiente, e a segunda para verificar se a distância da pessoa até a porta está dentro do intervalo desejado. Quando todas as *Premises* forem verdadeiras, a *Condition* ativará a execução da *Rule* através de sua *Action* que, neste exemplo, é composta por uma *Instigation* vinculada a um *Method* responsável por ligar a iluminação do ambiente e a outro *Method* para acionar o aviso sonoro.

A Fig. 2 apresenta o código dessa *Rule* em PON, na linguagem LingPON, que é uma das implementações do paradigma [12].

```
fbe Ambiente
attributes
  integer numPessoas 0
  integer distanciaPorta 0
  boolean lampada false
  boolean apito false
end_attributes
methods
  method AcendeLampada(lampada = true)
  method AcionaApito(apito = true)
end_methods
end_fbe
inst
  Ambiente ambiente
end_inst
rule Iluminacao
  condition
    subcondition Verifica
      premise temPessoa ambiente.numPessoas > 0 and
      premise pertoPorta ambiente.distanciaPorta <= 50
    end_subcondition
  end_condition
  action
    instigation Ilumina ambiente.AcendeLampada();
    instigation AvisoSonoro ambiente.AcionaApito();
  end_action
end_rule
```

Figura 2. Código de exemplo da *Rule* e da *FBE* em LingPON.

III. MATERIAIS E MÉTODOS

Para a avaliação do PON em uma aplicação de AAL, foi desenvolvido um sistema em linguagem C#, com a utilização do *framework* PON implementado nessa mesma linguagem. Este sistema permite o gerenciamento dos sensores e seus valores, representados respectivamente pela *FBE* e *Attributes* do PON, gerenciamento das *Rules* com suas *Conditions* e *Premises relacionadas*, *Actions* e *Instigations relacionadas*, e ainda, dos *Methods* que permitem a alteração dos atuadores no ambiente.

No mais, o sistema desenvolvido permite a execução e acompanhamento do ambiente simulado. Inclui-se nele, ainda, o desenvolvimento de uma aplicação do PON com o *framework* citado (em microcomputador), que recebe a definição das *Rules* referentes ao seu ambiente e inicia o monitoramento dos sensores e a alteração no ambiente conforme a execução das *Rules* definidas. Este fluxo é apresentado na Fig. 3.

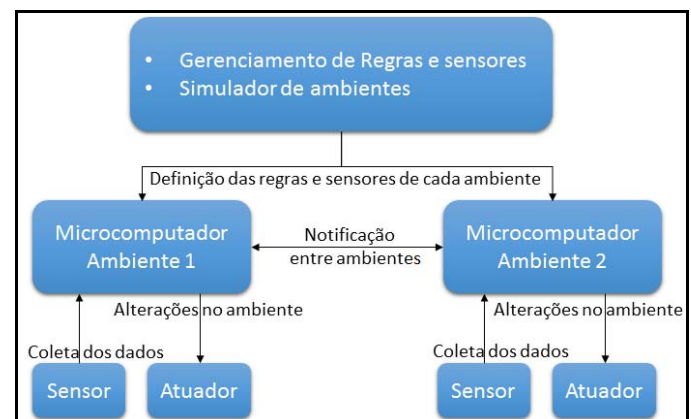


Figura 3. Fluxo dos dados no sistema de assistência à autonomia no domicílio.

Para a execução das *Rules* nos microcomputadores *Raspberry Pi 3*, foram utilizados os sensores de presença por infravermelho (PIR HC-SR501) e o sensor de distância por ultrassom (HC-SR04), juntamente com um atuador LED e um atuador do tipo alto-falante (módulo *buzzer* 5V passivo). A *Rule* consiste em verificar a presença de pessoas no ambiente e a distância dessa pessoa em relação à porta, acendendo a lâmpada à LED e acionando um apito no alto-falante quando a condição da *Rule* é satisfatória.

Na Tabela I, exibe-se uma lista de sensores e atuadores com suas principais aplicações, e que podem ser utilizados em sistemas de assistência à autonomia no domicílio. Os dispositivos são agrupados por cômodos (ambientes) e controlados por microcomputadores. Por exemplo, o *Raspberry Pi* é responsável pelo monitoramento e ações relacionadas aos eventos deste ambiente. Contudo, existem situações em que o estado de um sensor pode disparar uma ação em outro ambiente. Neste caso, é imprescindível uma aplicação distribuída que permita comunicação entre os vários microcomputadores de forma transparente e em tempo real.

TABELA I

EXEMPLO DE SENSORES E ATUADORES COM SUAS PRINCIPAIS APLICAÇÕES EM SISTEMAS DE ASSISTÊNCIA À AUTONOMIA NO DOMICÍLIO [4].

Dispositivos	Aplicação
Infravermelho	Identificação de movimento
Chaves magnéticas	Abertura e fechamento de portas
Luzes	Controle da iluminação
Ultrassom	Identificação de movimento
Microfone	Atividade no ambiente por meio do som
Giroscópio	Monitoramento da orientação
Glicosímetro	Monitoramento da glicose no sangue
Pressão	Monitoramento da pressão sanguínea
Eletrocardiograma	Monitoramento das atividades cardíacas
Oxímetro de pulso	Saturação do oxigênio no sangue
Térmico	Monitoramento da temperatura do corpo
Sensor de fumaça	Identificação de incêndios

A Fig. 4 apresenta a interface de gerenciamento do simulador com seus respectivos sensores/atuidores e *Rules*.

No gerenciamento dos sensores, é possível a inclusão e alteração de sensores e atuadores, definindo o nome, o valor inicial e o ambiente ao qual ele pertence; ambiente este representado pelo endereço IP do microcomputador *Raspberry Pi* do respectivo ambiente.

No gerenciamento das *Rules*, é possível sua criação e alteração, além do gerenciamento das partes da *Rule* como as *Conditions*, *Premises*, *Instigations* e *Methods*. As *Conditions* de uma *Rule* são definidas por meio de uma ou mais *Premises* e o operador lógico entre elas - E (*And*) ou Ou (*Or*).

No simulador, é possível criar *Subconditions* permitindo, assim, a formulação de várias *Premises* com operadores lógicos diferentes. O gerenciamento das *Premises* de uma *Condition* consiste na definição de uma expressão lógica que utiliza o valor de um ou mais sensores em sua formulação, por exemplo, Lâmpada = 1 (ligada), Temperatura < 10 (frio). O simulador também permite a definição de um tipo especial de *Premise* que, ao invés de avaliar a situação dos sensores, ela analisa a aprovação de uma determinada *Rule*, por exemplo, se a *Rule* da iluminação já foi acionada.

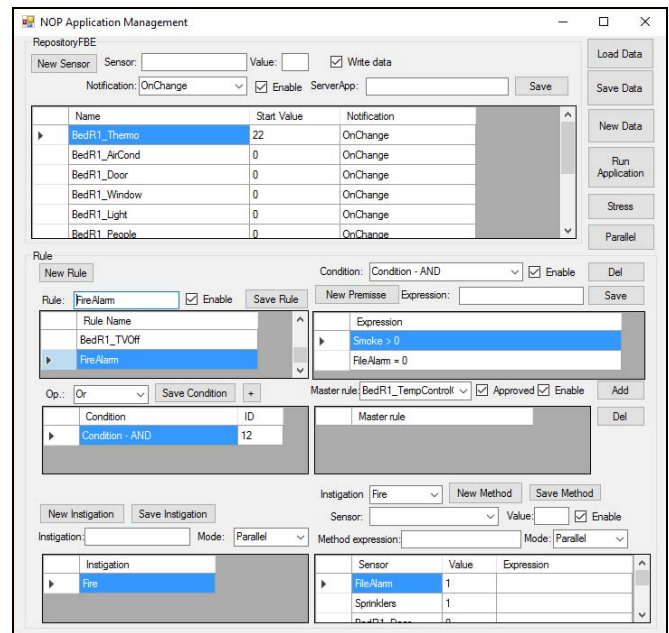


Figura 4. Interface do simulador de ambientes para gerenciamento dinâmico de sensores/atuidores, *Rules*, *Conditions*, *Premises*, *Instigations* e *Methods*.

As *Instigations* agrupam os *Methods* e podem ser executadas de forma sequencial ou paralela. No caso de um incêndio, acionado pela presença de fumaça no ambiente, ocorrerá, por exemplo, uma *Instigation* “fogo” que dispara o alarme e liga os chuveiros para apagar as chamas, mas poder-se-ia ter mais duas *Instigations*: uma para “abertura das portas” e “parada dos elevadores”, e outra para o “desligamento da energia”. Neste caso, o desligamento da energia deverá ser sequencial e não paralelo, para garantir que ao cortar a energia os elevadores estejam parados e com as portas abertas; caso contrário, a energia poderia ser desligada antes que as pessoas saíssem dos elevadores.

Os *Methods* são responsáveis por envio de dados aos atuadores ou até mesmo outros sistemas. Nos *Methods* define-se o atuador alvo e um valor ou uma expressão que calcule o valor a ser enviado para o atuador. De forma, semelhante às *Instigations*, os *Methods* também podem ser executados de forma sequencial ou paralela, por exemplo, uma *Rule* que verifica a temperatura do ambiente e liga o ar-condicionado, além de fechar a porta e a janela do ambiente. Caso o ar-condicionado seja ligado antes do fechamento da porta, o sistema pode disparar outra *Rule* que desliga o ar-condicionado em situações nas quais o mesmo esteja ligado e a porta do ambiente esteja aberta. Neste caso, o ar-condicionado poderia não ser ligado se os *Methods* fossem disparados paralelamente.

Para a execução das *Rules* e monitoramento dos sensores, desenvolveu-se uma interface de execução do ambiente onde é possível simular valores nos sensores e analisar as ações tomadas no ambiente, inclusive nas situações em que o sensor de um ambiente pode afetar o estado de outro, sendo possível, assim, testar o requisito de distribuição nos sistemas AAL. Durante a execução da simulação do ambiente, é possível alterar as *Rules*, *Premises*, *Instigations* e *Methods*, por meio da interface de gerenciamento, e acompanhar essas alterações na simulação em tempo real, sem a necessidade de qualquer

parada no sistema, testando, assim, a capacidade da gestão dinâmica dos sensores e *Rules* em um sistema em execução.

Na simulação do ambiente, foram utilizados os seguintes sensores/atuidores: termômetro, ar-condicionado, porta, janela, iluminação, presença de pessoa no ambiente, hora do dia, controle da TV, sensor de fumaça, alarme de incêndio e chuveiros antichamas (*sprinklers*). Com esses sensores/atuidores, foram criadas *Rules* que controlam a temperatura do ambiente, a iluminação, o aparelho de televisão e situações de incêndio. Um exemplo das *Premises* de uma dessas regras é o controle da iluminação do ambiente que acende as luzes se as mesmas estiverem apagadas, se houver alguém no ambiente e se o horário estiver compreendido entre 18 e 23 h. A Fig. 5 apresenta a estrutura das regras de climatização responsáveis pelo controle do ar-condicionado do ambiente.

Regra Ligar o ar-condicionado	
Se	
Há pessoas no ambiente E	
A temperatura do ambiente é maior que 23 graus E	
O ar-condicionado está desligado	
Então	
Ligue o ar-condicionado	
Regra Desligar o ar-condicionado	
Se	
O ar-condicionado está ligado E	
Não há pessoas no ambiente	
Então	
Desligue o ar-condicionado	

Figura 5. Definição das regras que controlam a temperatura do ambiente.

O resultado da simulação foi analisado de forma qualitativa, com o atendimento da aplicação frente aos requisitos dos sistemas AAL e de uma forma quantitativa, por meio de testes de *stress* e de notificações entre ambientes (distribuição). No teste de *stress*, utilizou-se várias simulações com 1, 5, 25, 50 e 100 ambientes, e em cada ambiente simulado, todos os onze sensores tiveram seus valores alterados de 0 a 30, de forma sequencial.

Ao término do teste, foram medidos o tempo total de processamento da simulação e contabilizados os números de notificações de sensores, análise de *Premises*, aprovações de *Rules* e execuções de *Methods* do ambiente, calculando, assim, as ações processadas em função do tempo, no microcomputador *Raspberry Pi*. No teste de notificações entre ambientes, foram enviadas 100 notificações de um microcomputador ao outro, ambos em uma mesma rede sem fio, utilizando os protocolos HTTP, TCP e UDP e, ao final do envio, foi medido o tempo total do teste em milissegundos. Para cada protocolo, foi executado um total de dez testes para se obter a média de tempo de execução no envio das notificações.

IV. RESULTADOS

O sistema desenvolvido para assistência à autonomia no domicílio com a utilização do paradigma orientado a notificações permitiu a inclusão de sensores, criação e

alteração de *Rules*, mudança nos operadores lógicos das *Conditions*, alteração nas expressões das *Premises*, criação e alteração nas *Instigations* e *Methods*, inclusive na forma de execução (sequencial ou paralela), e alteração nos valores enviados aos atuadores através dos *Methods*.

Os requisitos da computação senciente, como distribuição e paralelismo, foram comprovados no ambiente simulado por meio da distribuição das tarefas (notificações) entre todos os núcleos de processadores do equipamento, e nas situações em que a leitura de um sensor alterava o estado de outro ambiente por meio de notificações pela rede entre os microcomputadores.

Na Tabela II, apresenta-se o resultado do teste de *stress* e, conforme os dados apresentados, é possível calcular a média de 235 notificações de sensores por segundo, e a média de 415 avaliações de *Premises* por segundo, totalizando uma média de 650 execuções por segundo.

TABELA II
RESULTADO DO TESTE DE *STRESS*. AS COLUNAS EXIBEM RESPECTIVAMENTE, O NÚMERO DE AMBIENTES NA SIMULAÇÃO, O NÚMERO DE LEITURAS DE SENSORES, O TOTAL DE *PREMISES* VERIFICADAS E O TEMPO TOTAL DA SIMULAÇÃO EM SEGUNDOS.

Ambientes	Leituras	Premises	Tempos (s)
1	341	600	1,47
5	1.705	3.000	7,15
25	8.525	15.000	35,82
50	17.050	30.000	72,87
100	34.100	60.000	144,13

A Fig. 6 apresenta os resultados do teste de envio de cem notificações entre ambientes diferentes, nos quais os microcomputadores *Raspberry Pi* estavam conectados em uma mesma rede. As notificações foram enviadas utilizando os protocolos HTTP, TCP e UDP.

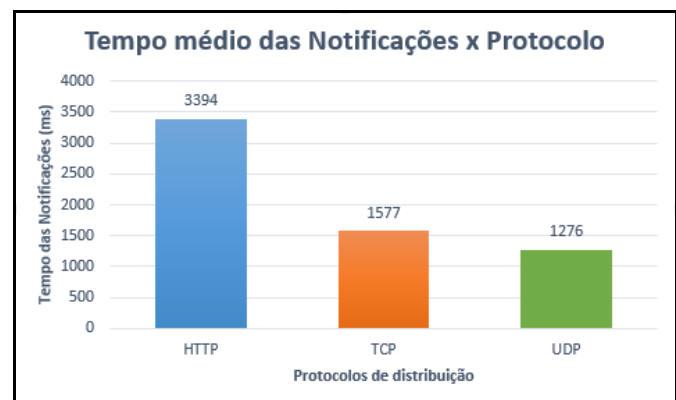


Figura 6. Resultado do teste de performance no envio de cem notificações entre ambientes, utilizando os protocolos HTTP, TCP e UDP.

V. DISCUSSÃO

O uso do PON em sistemas sencientes foi sugerido por Simão et al. [8], e este trabalho é a primeira implementação real do uso deste paradigma em ambientes inteligentes, gerando assim, resultados inéditos para a literatura. Os resultados obtidos foram comparados com as necessidades e

com os tempos esperados nas execuções desses sistemas em cenários reais, contornando, assim, a limitação nas comparações com os resultados de outros trabalhos, limitação esta, gerada pelo ineditismo desta aplicação.

Todas as alterações nos sensores e nas regras foram executadas com o simulador do ambiente rodando, permitindo assim que a gestão do sistema pudesse ser executada por uma equipe sem conhecimentos técnicos, tais como cuidadores, enfermeiros, familiares ou monitores de ambientes de assistência à autonomia de idosos.

A capacidade do sistema atuar de forma distribuída, com integração de sensores em ambientes diferentes, permite a utilização da tecnologia no monitoramento de ambientes compartilhados como lares, asilos, condomínios, bairros ou até mesmo cidades inteiras.

O teste de *stress*, em que se simulou a monitorização de vários ambientes com um único microcomputador, resultou em uma capacidade de processamento de 650 notificações por segundo, em média, sendo um tempo de resposta coerente com ambientes reais, levando-se em consideração que cada ambiente (cômodo) é controlado por um microcomputador. Os intervalos de notificação dos sensores ocorrem nos ambientes reais em intervalos maiores, garantindo, assim, a execução das *Rules* em menos de 1 s, fator fundamental, principalmente nas questões relacionadas à segurança, como incêndios, quedas do idoso, ou problemas de saúde como uma parada cardíaca ou respiratória.

No teste de notificações entre ambientes, todas as notificações foram recebidas pelo destino em ambos os protocolos testados, sendo o tempo de notificação do protocolo HTTP, em média, o dobro do tempo utilizado pelos protocolos TCP e UDP, um resultado esperado em virtude das particularidades de comunicação de cada protocolo. Contudo, a capacidade média de notificações nos protocolos HTTP, TCP e UDP foram, respectivamente, de 30, 63 e 78 notificações por segundo, sendo satisfatórios para envio de comando aos sensores que estão em outros ambientes. Ao comparar-se este resultado com o trabalho de Kruger e Hancke [13], identificou-se desempenho dez vezes melhor na comunicação de rede, ocasionado principalmente pela versão mais atual do microcomputador, pelo sistema operacional utilizado e pela otimização do pacote de notificação utilizado no PON.

V. CONCLUSÃO

A utilização de sistemas AAL que auxiliem o idoso na execução das principais atividades diárias, monitorando suas condições de saúde e as condições do seu ambiente, é fundamental para o aumento na qualidade de vida e nas condições de saúde da população idosa em diferentes níveis econômicos e sociais.

Para a disseminação da tecnologia, é importante que o sistema se adeque às mais variadas situações, perfis e condições, tanto do ambiente como das pessoas nele inseridas. Esta adequação deve ser intuitiva, transparente e rápida, além de permitir a incorporação de novas tecnologias de sensoriamento dos ambientes inteligentes.

A criação de uma aplicação que simula ambientes inteligentes para assistência à autonomia no domicílio, com a

utilização do Paradigma Orientado a Notificações, permite a avaliação da aderência dos requisitos da computação senciante no auxílio da execução das tarefas cotidianas dos idosos e demais pessoas com necessidade de autonomia. Neste cenário, a utilização do PON no desenvolvimento deste sistema contribuiu para uma implementação robusta e dinâmica, possibilitando, de uma forma natural, a criação e as adequações das *Rules* conforme o contexto e a situação, a execução de tarefas paralelas e a distribuição do sistema de forma lógica.

Os resultados obtidos no processamento das *Rules*, leitura dos sensores e notificações entre ambientes, com execuções em milissegundos, são adequados quando comparados às demandas dos cenários reais dos ambientes inteligentes, e demonstram a eficiência de aplicações para sistemas inteligentes desenvolvidos sob o PON.

Os recursos desta tecnologia, tais como extensibilidade, controle de dispositivos com microcomputadores (IoT) e monitoramento da condição de saúde, são diferenciais que permitem a criação de um sistema robusto e de encontro às necessidades da população que padece de auxílio em suas atividades diárias e aguardam que as soluções desta natureza, sejam realidades em suas vidas.

AGRADECIMENTOS

Os autores agradecem ao grupo de pesquisa do PON na UTFPR, particularmente os professores autores do PON, nomeadamente J. M. Simão e P. C. Stadzisz, por seu esforço e dedicação na evolução do paradigma e disseminação do conhecimento, e pelo entusiasmo e suporte proporcionado durante esta pesquisa.

REFERÊNCIAS

- [1] United Nations, *World Population Ageing, 1950-2050*. UN, no 207. 2002.
- [2] C. Ramos, *Ambient Intelligence – A State of the Art from Artificial Intelligence Perspective*, Progress in Artificial Intelligence, vol 18, pp. 285-295. 2007.
- [3] D. J. Cook, S. K. Das, *How Smart are our Environments? An Updated Look at the State of the Art*. Journal of Pervasive and Mobile Computing, vol 3, pp. 53-73. 2007.
- [4] P. Rashidi, A. Mihailidis. *A Survey on Ambient-Assisted Living Tools for Older Adults*. IEEE Journal of Biomedical and Health Informatics, vol 17, no 3, pp. 579-590. 2013.
- [5] J. M. Simão, R. F. Banaszewski, C. A. Tacla, P. C. Stadzisz, *Notification Oriented Paradigm (NOP) and Imperative Paradigm: A Comparative Study*, Journal of Software Engineering and Applications (JSEA), p.402-416, v.5, n.6, 2012. ISSN: 1945-3116. DOI 10.4236/jsea.2012.59083. <http://www.scirp.org/journal/PaperInformation.aspx?paperID=19842#abstract>
- [6] J. M. Simão, P. C. Stadzisz, *Inference Based on Notifications: A Holonic Meta-Model Applied to Control Issues*. IEEE Transactions on Systems, Man and Cybernetics, Part A. Vol. 39, Issue 1, Jan. 2009 Pg. 238-250. DOI 10.1109/TSMCA.2008.2006371. <http://ieeexplore.ieee.org/xpl/articleDetails.jsp?arnumber=4689369&newsearch=true&queryText=Stadzisz>
- [7] J. M. Simão, P. C. Stadzisz, *Notification Oriented Paradigm (NOP) — A Notification Oriented Technique to Software Composition and Execution*. Original title: *Paradigma Orientado a Notificações (PON) Uma Técnica de Composição e Execução de Software Orientada a Notificações*. 2008, Brasil. PEDIDO DE PATENTE: Privilégio de Inovação. Número do registro: PI08055181, data de depósito: 26/11/2008, INPI - Instituto Nacional da Propriedade Industrial. Universidade Tecnológica Federal do Paraná - UTFPR (Demanda Agência de Inovação, 2007). <http://www.patentesonline.com.br/paradigma-orientado-a-notificacoes-pom-uma-tecnica-de-composicao-e-execucao-de-software-234943.html>.

- [8] J. M. Simão; D. P. B. Renaux; R. R. Linhares; P. C. Stadzisz. *Evaluation of the Notification Oriented Paradigm applied to Sentient Computing*. In: 10th Workshop on Software Technologies for Future Embedded and Ubiquitous Systems (SEUS 2014) in 2014 IEEE 17th International Symposium on Object/Component-Oriented Real-Time Distributed Computing, 2014, Reno - Nevada - USA. 2014. v. 1555-0. p. 253-260. DOI: 10.1109/ISORC.2014.54.
- [9] J. M. Simão, *A Contribution to the Development of a HMS simulation tool and Proposition of a Meta-Model for Holonic Control*. Ph. D. Thesis. Graduate School in Electrical Engineering and Industrial Computer Science (CPGEI) at Federal University of Technology - Paraná (UTFPR, Brazil) and Research Center For Automatic Control of Nancy (CRAN) - Henry Poincaré University (UHP, France), 2005. Ph. D Thesis available on: http://arquivos.cpgei.ct.utfpr.edu.br/Ano_2005/teses/Tese_012_2005.pdf
- [10] R. R. Linhares, J. M. Simão and P. C. Stadzisz. *NOCA – A Notification-Oriented Computer Architecture*. IEEE Latin America Transactions, Vol. 13, Issue 5, May 2015.
- [11] D. Belmonte, R. R. Linhares, P. C. Stadzisz, J. M. Simão. *A new Method for Dynamic Balancing of Workload and Scalability in Multicore Systems*. IEEE Latin America Transactions, ISSN: 1548-0992. 2016.
- [12] C. A. Ferreira, *Linguagem e Compilador para o Paradigma Orientado a Notificações (PON): Avanços e Comparações*, Dissertação de Mestrado, PPGCA UTFPR, 2015.
- [13] C. P. Kruger, G. P. Hancke, *Benchmarking Internet of Things Devices*. 12th IEEE International Conference on Industrial Informatics, INDIN. pp. 611-616. 2014.



Rodrigo Nunes Oliveira nasceu em Campo Mourão-PR, em 1980. Possui graduação em Análise e Desenvolvimento de Sistemas pelo Centro Universitário Campos de Andrade (2006). E atualmente é estudante de mestrado na Universidade Tecnológica Federal do Paraná (UTFPR). É sócio na empresa GL2 Sistemas Ltda. Tem experiência na área de Ciência da Computação, com ênfase em Arquitetura de Sistemas de Computação, atuando principalmente nos seguintes temas: frameworks de desenvolvimento, ambientes virtuais em 3D, logística, Internet das Coisas e ambientes inteligentes para idosos. <http://lattes.cnpq.br/5147555676164143>.



Valmir Roth nasceu em Pitanga-PR, em 1987. Possui graduação em Sistemas de Informação pelas Faculdades SPEI (2011). Possui experiência em desenvolvimento de software na área industrial com foco em soluções para dispositivos móveis. E atualmente é estudante de mestrado na Universidade Tecnológica Federal do Paraná (UTFPR). <http://lattes.cnpq.br/8706103991317324>.



Alexandre Felippeto Henzen possui mestrado em Engenharia Elétrica e Informática Industrial pela Universidade Tecnológica Federal do Paraná (2003). Atualmente é estudante de Doutorado em Engenharia Biomédica na Universidade Tecnológica Federal do Paraná (UTFPR) e diretor de tecnologia na KORP INFORMÁTICA LTDA. <http://lattes.cnpq.br/3835243107251008>.



Jean Marcelo Simão recebeu o grau de Técnico em Informática pelo Colégio Estadual do Paraná (CEP) e, em 1998, o grau de Bacharel em Informática pela Universidade Estadual de Ponta Grossa (UEPG). Em 2001, ele obteve o grau de M. Sc. do Curso-Programa de Pós-graduação em Engenharia Elétrica e Informática Industrial (CPGEI) do então Centro Federal de Educação Tecnológica Federal do Paraná (CEFET-PR), situado em Curitiba - PR (Brasil), atualmente Universidade Tecnológica Federal do Paraná (UTFPR). Em junho de 2005, ele obteve o grau de Doutor, depois de uma dupla tese de doutoramento, nos domínios de Informática Industrial no CPGEI/UTFPR e de Engenharia da Computação & Automática na Universidade Henry Poincaré (UHP) - Centro de Pesquisa em Automática de Nancy (CRAN), estes dois situados na França. Depois, em 2005/2006, ele desenvolveu atividades de ensino na UHP, atual Université de Lorraine (UL), e atividades de pesquisa no CRAN em um contexto Pós-doutoral. Atualmente, desde 2006, ele é Professor na UTFPR. Suas atividades de ensino são em ciência e engenharia da computação, enquanto seus interesses de pesquisa incluem ciência da computação e paradigmas de desenvolvimento. <http://lattes.cnpq.br/3593420323268103>.



Emilio Carlos Gomes Wille é graduado em Engenharia Elétrica - Eletrônica Industrial e Telecomunicações (1988), mestre em Engenharia Elétrica e Informática Industrial pela Universidade Tecnológica Federal do Paraná - UTFPR (1991) e doutor em Engenharia Eletrônica e Telecomunicações pelo Politecnico di Torino - Itália (2004). É professor titular da UTFPR, atuando no Curso de Engenharia Industrial Elétrica e Telecomunicações, e no Programa de Pós-Graduação em Engenharia Elétrica e Informática Industrial. Tem experiência na área de Engenharia Elétrica, com ênfase em Redes de Telecomunicações, principalmente em modelamento e determinação de desempenho de sistemas de computação e telecomunicações, TCP/IP, redes sem fio, teoria de filas, pesquisa operacional e otimização, e códigos corretores de erro. <http://lattes.cnpq.br/7042348032717400>.



Percy Nohama é graduado em Filosofia pela Universidade Federal do Paraná (1980), em Licenciatura em Eletrônica pela Universidade Tecnológica Federal do Paraná (1986), especialização em Metodologia do Ensino Superior pela Universidade Federal do Rio Grande do Sul (1982), e em História do Pensamento Contemporâneo pela Pontifícia Universidade Católica do Paraná (1986), mestrado (1992) e doutorado (1997) em Engenharia Elétrica pela Universidade Estadual de Campinas. Atualmente, é professor titular da Pontifícia Universidade Católica do Paraná e professor voluntário da Universidade Tecnológica Federal do Paraná. Atua na área de engenharia de reabilitação e tecnologias assistivas. <http://lattes.cnpq.br/5055126579468463>.